Zadanie Numeryczne NUM4

- Jakub Pawlusek -

15 grudzień 2024

1. Treść zadania:

7. **(zadanie numeryczne NUM4)** Zadana jest macierz

$$\mathbf{A} = \begin{pmatrix} 5 & 3 & 1 & 1 & \dots & 1 & 1 & 1 & 1 \\ 1 & 5 & 3 & 1 & \dots & 1 & 1 & 1 & 1 \\ 1 & 1 & 5 & 3 & \dots & 1 & 1 & 1 & 1 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 1 & 1 & 1 & 1 & \dots & 1 & 5 & 3 & 1 \\ 1 & 1 & 1 & 1 & \dots & 1 & 1 & 5 & 3 \\ 1 & 1 & 1 & 1 & \dots & 1 & 1 & 1 & 5 \end{pmatrix}$$

oraz wektor $\mathbf{b} \equiv (2, \dots, 2)^T$. Macierz $\mathbf{A}$ ma liczby 5 na diagonali, 3 na pierwszej pozycji nad diagonalą, a pozostałe elementy są równe 1. Wymiar macierzy ustalamy na $N = 120$.

- Rozwiąż numerycznie równanie $\mathbf{Ay} = \mathbf{b}$, stosując odpowiednią metodę. **Uwaga:** algorytm należy go zaimplementować samodzielnie.
- Sprawdź swój wynik przy użyciu procedur bibliotecznych lub pakietów algebry komputerowej.
- Potraktuj N jako zmienną i zmierz czas potrzebny do uzyskania rozwiązania w funkcji N. Wynik przedstaw na wykresie. Czy wykres jest zgodny z oczekiwaniami?

2. Wstęp:

2.1. Cel zadania

Naszym celem jest rozwiązanie numerycznie równania Ay = b, gdzie macierz A i wektor b są podane w treści zadania. Po zastosowaniu wybranej metody potem porównamy wyniki z rozwiązaniem jakie daje nam biblioteka numpy oraz sprawdzimy z jaka złożonością będzie działać nasza wybrana metoda. Na podstawie wyników w formie liczbowej i wizualnej na wykresach dokonamy analizy co się stało.

2.2. Metody

Do rozwiązania tego problemu zaimplementowałem program w języku Python3, który rozwiązuje nam nasze równanie Ay = b na dwa sposoby: naszym algorytmem oraz biblioteką numpy.

Jak dokładniej opisze w założeniach naszą wybraną metodą do zadania będzie algorytm Shermana-Morrisona.
Do niego mamy napisaną funkcję: `def sherman_morrison(M, b, n):`
Dodatkowo do liczenia wartości wektorów z i q (patrz założenia) używamy tzw. backwards substitution.
A testowe rozwiązanie przy użyciu biblioteki numpy korzysta z funkcji takich jak
np.diag(), np.ones() i np.full()

Czas działania mierzymy biblioteka “time”, a wykresy są stworzone przy użyciu biblioteki matplotlib.pyplot.

2.3. Założenia

Na początek możemy zauważyć, że naszą macierz A da się przekształcić w wygodniejszą dla nas formę czyli wstęgową. Robimy to w następujący sposób:

$$A = \begin{pmatrix} 5 & 3 & 1 & \cdots & 1 \\ 1 & 5 & 3 & \cdots & 1 \\ 1 & 1 & 5 & \cdots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \cdots & 5 \end{pmatrix} = \begin{pmatrix} 4 & 2 & 0 & \cdots & 0 \\ 0 & 4 & 2 & \cdots & 0 \\ 0 & 0 & 4 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 4 \end{pmatrix} + \begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix}$$

Można powiedzieć, że “wyjęliśmy” macierz wypełnioną jedynkami lub po prostu, że odjęliśmy 1 od każdego elementu przy okazji dodając różnicę. Nową macierz “główną” oznaczę A’.

Z tego otrzymaliśmy więc macierz wstęgową, która wygląda następująco:

$$\begin{pmatrix} 4 & 2 \\ 4 & 2 \\ \vdots & \vdots \\ 4 & 2 \\ 4 & 0 \end{pmatrix}$$

Dzięki takiej zmianie obliczenia będą znacznie przyspieszone, bo pomijamy elementy zerowe.

Macierz wypełnioną jedynkami za to możemy rozdzielić na wektory uv^T ,gdzie $u = v = [1,...,1]^T$.

To oznaczenie nie pojawia się bez powodu, ponieważ teraz przechodzimy do metody, która została wybrana do rozwiązania tego zadania.

Jest to algorytm Shermana - Morrisona. Pozwoli on nam rozwiązać ten problem w czasie O(N). Możliwe jest to przez to, że ta metoda pozwala nam obliczyć odwrotność macierzy A jeśli znamy odwrotność A’.

Możemy zapisać ze wcześniejszych założeń, że $A = A' + uv^T$ i korzystając ze wzoru Shermana-Morrisona otrzymamy:

$A^{-1} = (A' + uv^T)^{-1} = A'^{-1} - \dfrac{(A'^{-1}uv^TA'^{-1})}{1 + v^TA'^{-1}u}$ skorzystanie z tego pozwoli na szybkie i tanie wyliczenie odwrotności macierzy A.

Robimy to ponieważ, żeby obliczyć Ay=b jeśli przemnożymy to równanie przez A^{-1} to otrzymamy:

$y = A^{-1}b = (A' + uv^T)b = (A'^{-1} - \dfrac{(A'^{-1}uv^TA'^{-1})}{1 + v^TA'^{-1}u})b.$

Algorytm rozwiązywania metodą Shermana-Morrisona polega na 3 krokach:

1) Rozwiązujemy równanie: A*z = b

2) Rozwiązujemy równanie: A*q = u

3) Liczymy: $w = z - \dfrac{v^Tz}{1 + v^Tq}q$

Takim sposobem otrzymamy rozwianie do zadania.

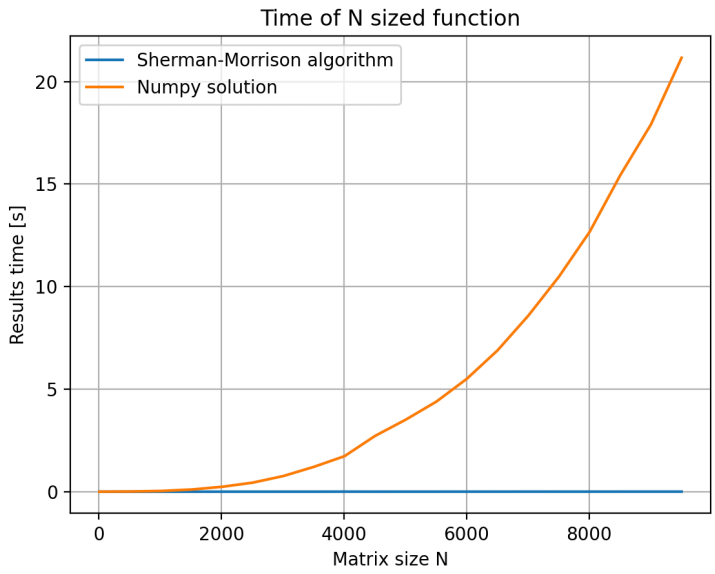
3. Wyniki:

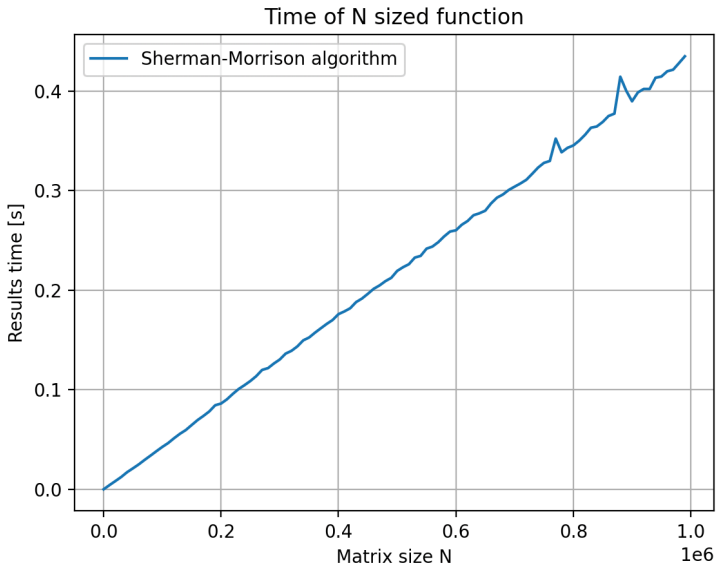
Forma liczbowa (wyniki równania) :

Sherman-Morrison test results:

[0.0158311345646438, 0.0158311345646438, 0.0158311345646438, 0.0158311345646438,

Forma graficzna (pomiar czasu) :





4. Analiza wyników:

Na podstawie analizy wyników możemy wywnioskować kilka rzeczy:

Jeśli chodzi o wyniki dla równania można zauważyć, że wyniki dla ręcznej implementacji i bibliotecznych obliczeń są do siebie podobne co oznacza, że zaimplementowany algorytm Shermana - Morrisona jest poprawny i działa jak należy.

Widać też, że wyniki z ręcznie zaimplementowanego algorytmu mają większą dokładność niż obliczone przy użyciu biblioteki Numpy.

Teraz przeanalizujmy wykresy.

Pierwszy wykres przedstawia zależność czasową dla obu algorytmów. Jak widać odrazu rzuca się w oczy to, że algorytm Shermana - Morrisona jest dużo szybszy niż algorytm stworzony przez bibliotekę Numpy.

Wnioskując z naszych założeń to spodziewaliśmy się tej różnicy ponieważ to tego rodzaju problemów wzór SM jest bardzo efektywny.

Jak widać po drugim wykresie udowodnione jest to, że ten algorytm działa w czasie O(N) co jest bardzo zadowalającą złożonością.